Labo 17 — Réponses commentées

Chaque réponse suit le même schéma : la réponse, le mécanisme, la nuance ou le piège, un exemple vérifiable au terminal.

Question 1 — Chart, release, revision

Réponse. Le **chart** est le paquet : des templates, des valeurs par défaut et des métadonnées, dans un dossier ou un `.tgz`. Une **release** est une installation de ce chart sous un nom, dans un namespace (`stack` dans `lab17`) : on peut installer le même chart plusieurs fois sous des noms différents. Une **revision** est une version numérotée de la release : chaque `install`, `upgrade` ou `rollback` en crée une. Helm stocke chaque revision (chart + valeurs + manifeste rendu) dans un Secret `sh.helm.release.v1.stack.v<n>` du namespace. Comme tout est dans l'API server, un client seul, avec un kubeconfig, retrouve l'historique et peut réappliquer une revision : pas besoin de Tiller ni de base de données.

Pourquoi. Helm 3 a remplacé le serveur Tiller par ce stockage dans le cluster, avec les droits RBAC de l'utilisateur.

Nuance. Par défaut Helm garde 10 revisions (`--history-max`). Supprimer ces Secrets à la main détruit l'historique ; `helm uninstall` les supprime aussi (sauf `--keep-history`).

Exemple.

```
kubectl get secrets -n lab17 -l owner=helm      # sh.helm.release.v1.stack.v1, .v2, …
helm history stack -n lab17
```

Question 2 — `--set` et valeurs oubliées

Réponse. `helm get values` affiche les **valeurs fournies par l'utilisateur** à la dernière revision : seulement `api.message: v2`. Après la troisième commande, l'API tourne avec **2 réplicas** (la valeur de `values.yaml`), pas 3 : chaque `helm upgrade` repart des valeurs par défaut du chart plus ce qu'on lui donne **à cette commande**. Pour cumuler : `helm upgrade stack ./stack --reuse-values --set api.message=v2`, ou redonner les deux `--set`, ou — mieux — tout mettre dans un `values-<env>.yaml` passé à chaque fois avec `-f`.

Pourquoi. Les `--set` ne sont pas persistants par eux-mêmes ; ils ne sont « mémorisés » que dans la revision, et un nouvel upgrade n'en hérite qu'avec `--reuse-values`.

Nuance. `--reuse-values` est piégeux à son tour : il ignore les nouvelles valeurs par défaut d'un chart mis à jour. La pratique d'entreprise : un fichier de valeurs par environnement, versionné, et jamais de `--set` « à la main » en production.

Exemple.

```
helm upgrade stack ./stack --set api.replicas=3 ; helm upgrade stack ./stack --set
api.message=v2
kubectl get deployment api -o jsonpath='{.spec.replicas}'    # 2
helm get values stack                                        # api: message: v2
```

Question 3 — Helm ne trouve pas le cluster

Réponse. `helm` cherche un kubeconfig à l'endroit standard (`~/.kube/config` ou `$KUBECONFIG`) ; il n'y en a pas, donc il tente l'adresse par défaut `localhost:8080`, où rien n'écoute. Correction : `export KUBECONFIG=/etc/rancher/k3s/k3s.yaml` (ou copier ce fichier dans `~/.kube/config`). Le `kubectl` de k3s n'a pas ce problème parce qu'il est **embarqué dans le binaire k3s**, qui connaît son propre kubeconfig.

Pourquoi. Helm est un client générique, comme Lens, k9s ou un `kubectl` installé séparément ; tous lisent `KUBECONFIG`.

Nuance. Le fichier est lisible grâce à `--write-kubeconfig-mode 644` (labo 11) ; sur un cluster d'entreprise, le kubeconfig vous est fourni par l'équipe plateforme et contient des identifiants : ne le commitez jamais.

Exemple.

```
export KUBECONFIG=/etc/rancher/k3s/k3s.yaml ; helm list -A    # traefik, traefik-crd
```

Question 4 — L'annotation `checksum/config`

Réponse. À `helm upgrade --set api.message=autre`, le template `configmap.yaml` est rendu avec le nouveau message ; son `sha256sum` change ; l'annotation du template de Pod change donc aussi ; comme le template de Pod a changé, le Deployment lance un **rolling update** et les nouveaux Pods lisent la nouvelle ConfigMap. Sans l'annotation, la ConfigMap serait mise à jour mais le Deployment, inchangé, garderait ses Pods avec l'ancien environnement (labo 14) : il faudrait un `kubectl rollout restart`. Cas non souhaité : la ConfigMap contient quelque chose qui varie sans raison fonctionnelle (un horodatage, un commentaire, l'ordre des clés) : chaque upgrade redéploie pour rien.

Pourquoi. Le Deployment ne surveille que son propre `spec.template` ; l'annotation y « importe » le contenu de la ConfigMap sous forme d'empreinte.

Nuance. La même technique vaut pour les Secrets (`secret.yaml`). Une alternative est un contrôleur dédié (Reloader) qui observe les ConfigMaps et redémarre les Deployments qui les montent.

Exemple.

```
kubectl get deployment api -o jsonpath='{.spec.template.metadata.annotations.checksum/config}'
; echo
helm upgrade stack ./stack --reuse-values --set api.message=autre
kubectl get deployment api -o jsonpath='{.spec.template.metadata.annotations.checksum/config}'
; echo   # différent
```

Question 5 — Rollback et PVC

Réponse. Après `helm rollback stack 1`, l'historique compte **quatre** revisions : 1, 2, 3 `superseded` et 4 `deployed`, décrite `Rollback to 1`. Helm n'efface rien : revenir en arrière est une **nouvelle revision** dont le contenu est celui de la 1. Après `helm uninstall`, `kubectl get pvc` montre toujours `data-db-0` `Bound` : Helm supprime les objets qu'il a créés, et la PVC a été créée par le **StatefulSet** via `volumeClaimTemplates`, pas par Helm. C'est volontaire : désinstaller une

release ne doit pas détruire une base. Les données disparaissent avec `kubectl delete pvc data-db-0` ou la suppression du namespace.

Pourquoi. Même logique que `podman compose down` sans `-v` et que `kubectl delete statefulset` (labo 15).

Nuance. Un chart peut déclarer des hooks ou l'annotation `helm.sh/resource-policy: keep` pour protéger d'autres objets. Et `helm rollback` ne ramène pas le contenu d'une base : il ne gère que des manifestes.

Exemple.

```
helm history stack | tail -1     # 4  …  deployed  …  Rollback to 1
helm uninstall stack ; kubectl get pvc    # data-db-0   Bound
```

Question 6 — Helm ou Kustomize

Réponse. (a) PostgreSQL et Prometheus : **Helm**, avec les charts officiels (Bitnami, prometheus-community) et un `values.yaml` par environnement : ces logiciels sont paramétrés par des centaines d'options, maintenus par d'autres, et versionnés comme des paquets. (b) Quatre microservices maison dans trois environnements via Argo CD : **Kustomize** : une base par service, un overlay par environnement, le tout lisible en YAML pur dans Git, sans templates à déboguer, et Argo CD l'applique nativement. Ils cohabitent parce que chaque équipe fait les deux : installer des produits tiers et déployer les siens ; Argo CD sait d'ailleurs rendre un chart Helm, et `helm template | kustomize` permet de patcher un chart public sans le forker.

Pourquoi. Helm optimise la **distribution** (paquet, version, dépôt) ; Kustomize optimise la **lisibilité** de variantes d'un YAML que vous possédez.

Nuance. Un chart maison très simple (comme `stack/`) et un overlay Kustomize se valent ; la querelle Helm/Kustomize est surtout une question d'équipe. Le vrai critère : qui maintient le YAML, et combien d'environnements.

Exemple.

```
helm template stack ./stack -f stack/values-prod.yaml | grep -c '^kind:'    # 11 objets rendus
kubectl kustomize kustomize/overlays/prod | grep -c '^kind:'                 # 3
```

Question 7 — Label absent des Pods

Réponse. Le transformateur `labels` de Kustomize, avec `includeSelectors: false` et rien d'autre, n'ajoute le label qu'aux **métadonnées des objets** (Deployment, Service), pas au `spec.template` des Pods. Il faut `includeTemplates: true` pour qu'il atteigne aussi les templates de Pod. `includeSelectors: true` ajouterait le label **aux selectors** (`spec.selector.matchLabels` du Deployment, `spec.selector` du Service) : or le selector d'un Deployment est **immuable** ; sur un Deployment existant, l'`apply` échouerait (`field is immutable`), et il faudrait le supprimer et le recréer.

Pourquoi. Kustomize distingue trois cibles pour un label : l'objet, son template de Pod, ses selectors ; l'ancien `commonLabels` touchait les trois, d'où sa dépréciation.

Nuance. Un label de *selector* doit rester stable toute la vie d'un Deployment ; un label d'*information* (`env`, `team`, `version`) va dans l'objet et le template, jamais dans le selector.

Exemple.

```
labels:
  - pairs: { env: prod }
    includeSelectors: false
    includeTemplates: true
```

```
kubectl get pods -n lab17-prod -L env     # ENV prod
```

Question 8 — Deux digests, une image

Réponse. Le digest local (`sha256:e1b2…`) est l'empreinte du **manifeste tel que Podman le tient localement** ; au push, Podman **recompresse** les couches et produit un manifeste différent pour le registry, dont l'empreinte est `sha256:b820…` : c'est celui que `--digestfile` écrit et que le nœud voit dans `imageID` après le pull. Les deux désignent le même contenu, mais seul le digest **du registry** est celui qu'un cluster peut tirer : c'est lui qu'on épingle (`image: localhost:5000/api@sha256:b820…`). Un tag ne suffit pas parce qu'il est mobile : un nouveau push sur `api:1.1` change ce que `1.1` désigne, silencieusement.

Pourquoi. Un digest est le hachage d'un manifeste précis (liste de couches compressées + config) ; changer la compression change le manifeste, pas l'image une fois décompressée (même `IMAGE ID` , labo 10).

Nuance. `podman pull` depuis le registry puis `podman image inspect` donne cette fois `b820…` : le digest local devient celui du registry. `skopeo inspect docker://…` interroge le registry sans rien tirer.

Exemple.

```
podman push --tls-verify=false --digestfile digest.txt localhost:5000/api:1.1 ; cat digest.txt
kubectl get pods -l app=api -o jsonpath='{.items[0].status.containerStatuses[0].imageID}'    #
…@sha256:b820…
```

Question 9 — HTTP pour un registry non local

Réponse. containerd (comme Podman) tolère le **HTTP en clair uniquement pour `localhost` / `127.0.0.1`** ; pour tout autre hôte il exige TLS, d'où `server gave HTTP response to HTTPS client` . Sur k3s, on déclare le registry dans `/etc/rancher/k3s/registries.yaml` :

```
mirrors:
  "registry.interne:5000":
    endpoint: ["http://registry.interne:5000"]
```

puis `sudo systemctl restart k3s` : k3s régénère la configuration de containerd au démarrage. Sur un cluster managé, on ne touche pas aux nœuds : le registry **doit** être en TLS (certificat reconnu), et les identifiants vont dans un Secret `kubernetes.io/dockerconfigjson` référencé par `imagePullSecrets` dans le Pod (ou le ServiceAccount).

Pourquoi. Le pull est fait par le kubelet/containerd du nœud, avec la configuration du nœud ; l'API server n'y participe pas.

Nuance. Le même fichier sert aux miroirs (`"docker.io": endpoint: [...]`) et à l'authentification (`configs: … auth: username/password`). Un registry HTTP, même interne, est une exception de labo : en entreprise, TLS partout.

Exemple.

```
kubectl create secret docker-registry reg --docker-server=registry.interne --docker-username=ci
--docker-password=…
kubectl patch serviceaccount default -p '{"imagePullSecrets":[{"name":"reg"}]}'
```

Question 10 — `helm upgrade --install --wait`

Réponse. `--install` rend la commande **idempotente** : première exécution = `install`, suivantes = `upgrade`, même ligne dans tous les cas. `--wait` fait attendre que tous les Pods soient `Ready` (probes du labo 12), les Services dotés d'endpoints, les PVC liées ; `--timeout 5m` borne cette attente, après quoi la release passe `failed` et le job échoue : la CI **sait** que le déploiement n'a pas convergé. Le runner devrait avoir un kubeconfig lié à un **ServiceAccount** avec un RoleBinding `edit` sur le seul namespace cible (labo 16) : jamais le kubeconfig `cluster-admin` de k3s. En cas d'échec : `helm rollback api` (ou `--atomic`, qui le fait seul), puis alerter.

Pourquoi. Sans `--wait`, `helm upgrade` rend la main dès que l'API server a accepté les objets, bien avant que les Pods tournent.

Nuance. `--wait` ne détecte que ce que les probes révèlent : une readiness honnête (labo 12) est la condition de sa valeur. Et un Secret pour le kubeconfig du runner est un secret comme un autre : rotation, portée minimale.

Exemple.

```
helm upgrade --install stack ./stack -n lab17 --create-namespace --wait --timeout 2m --atomic
helm history stack      # failed → rollback automatique avec --atomic
```

Question 11 — GitOps

Réponse. En CI « push », un job exécute `kubectl` / `helm` vers le cluster : il faut lui confier des identifiants, et l'état réel du cluster finit par diverger de ce qui est dans Git (un `kubectl edit` d'urgence, un `helm rollback` manuel). En GitOps « pull », un agent **dans** le cluster (Argo CD, Flux) lit un dépôt Git et applique en continu ce qu'il décrit ; la CI ne fait que construire, pousser l'image et **modifier le tag dans Git**. Avantages : chaque changement est un commit (audit, revue par *merge request*), le rollback est un `git revert`, la **dérive** est détectée et corrigée (ou signalée). Conséquence : la CI n'a plus aucun accès au cluster. C'est la boucle du labo 11 — état voulu vs état réel — appliquée au dépôt Git entier.

Pourquoi. Le modèle déclaratif rend possible une source de vérité externe ; le contrôleur Kubernetes réconcilie des objets, l'agent GitOps réconcilie des dépôts.

Nuance. Les secrets ne vont pas en clair dans Git : Sealed Secrets, SOPS ou External Secrets (labo 14). Et un changement « d'urgence » fait à la main sera écrasé par l'agent : c'est le but.

Exemple.

```
git commit -am "api: 1.1 -> 1.2 en prod" && git push     # c'est tout le déploiement
kubectl get applications -n argocd                       # Synced / OutOfSync (Argo CD)
```

Question 12 — Vers un cluster d'entreprise

Réponse. Sur EKS/AKS/GKE : `Service: LoadBalancer` obtient un load balancer cloud avec une IP publique (et un coût) au lieu de ServiceLB ; `local-path` n'existe pas — la StorageClass par défaut est un disque réseau (`gp3`, `managed-csi`, `pd-balanced`), ce qui permet au Pod de changer de nœud ; l'Ingress controller doit être installé ou remplacé (nginx, ALB, GKE Ingress) ; l'identité passe par IAM/Entra/Google et des RBAC posés par la plateforme. Sur OpenShift s'ajoutent les **Routes** (à la place ou en plus des Ingress), les **SCC** qui refusent par défaut un conteneur root (votre `USER` du labo 05 devient obligatoire, et l'UID est imposé) et `oc`. Ne changent pas : Pods, Deployments, Services, ConfigMaps/Secrets, PVC, probes, RBAC, le chart lui-même. Pour rester portable : ne pas coder de StorageClass en dur (laisser la défaut, ou une valeur), rendre l'Ingress et ses annotations paramétrables, ne jamais supposer un port de nœud, tourner non-root, et déclarer `resources`.

Pourquoi. L'API est standard ; les intégrations (réseau, stockage, identité) sont fournies par chaque plateforme derrière les mêmes objets.

Nuance. `hostPath`, `hostPort`, `NodePort` et tout ce qui suppose « un seul nœud » cassent en premier. Testez le chart avec `helm template -f values-cloud.yaml` avant d'avoir le cluster.

Exemple.

```
# values.yaml — portable
db: { storageClass: "" }        # "" = StorageClass par défaut du cluster
web: { ingress: { enabled: true, className: "", annotations: {} } }
```

Question 13 — `latest` + `rollout restart`

Réponse. Reproductibilité : `latest` désigne une image différente à chaque push ; deux Pods du même Deployment peuvent tourner deux versions, et personne ne sait laquelle. Rollback : `rollout undo` ramène un template identique (`latest`), donc **la même image cassée** ; sans tag, il n'y a rien vers quoi revenir. Audit : aucun commit, aucune revision Helm ne dit ce qui tourne ; `imageID` est le seul témoin. Sécurité : `Always` + `latest` signifie que quiconque peut pousser sur ce tag déploie en production sans passer par la CI. Version correcte : la CI construit `api:<sha-git>` (immuable), pousse, puis `helm upgrade --install … --set api.image.tag=<sha> --wait --atomic` (ou modifie le tag dans Git pour Argo CD) ; en production, le digest ; `imagePullPolicy: IfNotPresent` suffit alors, puisque le tag ne bouge jamais.

Pourquoi. Un déploiement doit être une fonction de ses entrées (chart + valeurs + digest) ; `latest` retire l'entrée la plus importante.

Nuance. `latest` reste acceptable… pour un essai local sur k3s, jamais au-delà. Et `rollout restart` est utile — pour recharger une config, pas pour livrer une version.

Exemple.

```
helm upgrade --install stack ./stack --set api.image.tag=$(git rev-parse --short HEAD) --wait --atomic
helm history stack      # chaque revision nomme une image précise
```